

COMPANION
to

ChatGPT Teaches TikZ

Appendix D: TikZ Syntax at a Glance

Contents

D.1	Minimal Document and Picture	3
D.2	Paths	4
D.3	Coordinates	5
D.4	Nodes, Names, and Anchors	6
D.5	Connections and Labels	7
D.6	Line, Shape, Fill, and Transparency Options	8
D.7	Reusable Styles	9
D.8	Styles with Parameters	10
D.9	Custom Keys	11
D.10	Loops and Computed Values	12
D.11	Scopes and Transformations	13
D.12	Clipping and Layers	14
D.13	Reusable Pictures with <code>pic</code>	15
D.14	Matrices and Trees	16
D.15	Intersections	17
D.16	Decorations	17
D.17	Externalization	19
D.18	Libraries Used Most Often in This Book	20
D.19	Task-to-Syntax Quick Finder	20
D.20	Final Advice	22

This appendix is a compact syntax reference for the constructions used throughout the book. Appendix A answers the question “What does this command do?” Appendix B answers “Which library supplies the feature?” Appendix C answers “Why did this fail?” The present appendix answers a different question: “What is the syntax again?”

The examples are intentionally small. They are patterns to copy, adapt, and combine, not substitutes for the explanations in the chapters.

Principle

When you remember the structure but forget the punctuation, copy the smallest pattern that matches the task and change only the names, coordinates, labels, or options that matter.

D.1 Minimal Document and Picture

A complete minimal document is

Code

```
\documentclass{article}
\usepackage{tikz}

\begin{document}
\begin{tikzpicture}
  \draw (0,0) -- (3,1);
\end{tikzpicture}
\end{document}
```

The general picture structure is

Code

```
\begin{tikzpicture}[options]
  % TikZ commands
\end{tikzpicture}
```

Drawing commands end with semicolons.

D.2 Paths

The basic path patterns are

Task	Syntax
Straight segment	<code>\draw (A) -- (B);</code>
Several segments	<code>\draw (A) -- (B) -- (C);</code>
Closed polygon	<code>\draw (A) -- (B) -- (C) -- cycle;</code>
Rectangle	<code>\draw (0,0) rectangle (3,2);</code>
Circle	<code>\draw (0,0) circle (1cm);</code>
Ellipse	<code>\draw (0,0) ellipse (2cm and 1cm);</code>
Flexible connection	<code>\draw (A) to (B);</code>
Curved connection	<code>\draw (A) to[bend left=20] (B);</code>
Bézier curve	<code>\draw (A) .. controls (C1) and (C2) .. (B);</code>
Orthogonal path	<code>\draw (A) - (B);</code> or <code>\draw (A) - (B);</code>
Invisible path	<code>\path ... ;</code>
Clip to path	<code>\clip ... ;</code>

Options follow the command name.

Code

```
\draw[very thick,dashed,blue] (A) -- (B);
```

D.3 Coordinates

Ordinary Cartesian coordinates have the form

Code

```
(2,1.5)
```

A named coordinate is

Code

```
\coordinate (A) at (2,1.5);
```

The `calc` library adds coordinate calculations.

Code

```
\usetikzlibrary{calc}
```

Common patterns are

Relationship	Syntax
Midpoint of A and B	$\$(A)!0.5!(B)\$$
Fraction t from A to B	$\$(A)!t!(B)\$$
Fixed distance toward B	$\$(A)!1cm!(B)\$$

Continued on next page

Relationship	Syntax
Shift a point	$\$(A)+(1,0)\$$
Coordinate difference	$\$(A)-(B)\$$
Use x of A, y of B	$(A - B)$
Use y of A, x of B	$(A - B)$
Perpendicular direction	$\$(A)!t!90:(B)\$$

D.4 Nodes, Names, and Anchors

The general node form is

Code

```
\node[options] (name) at (coordinate) {text};
```

Every part except the visible text can be omitted when it is not needed.

Code

```
\node at (0,0) {Hello};
\node[draw,circle] (A) at (2,0) {$A$};
```

Anchors select a point on a node.

Code

```
\draw (A.east) -- (B.west);
\node[anchor=north west] at (0,0) {Text};
```

Relative placement uses the `positioning` library.

Code

```
\usetikzlibrary{positioning}

\node (A) {$A$};
\node[right=of A] (B) {$B$};
\node[below=12mm of A] (C) {$C$};
```

D.5 Connections and Labels

A straight labeled connection is

Code

```
\draw[-{Stealth}]
  (A) -- node[midway,above] {$f$} (B);
```

Useful label positions include

Code

```
node[near start,above] {$f$}
node[midway,above] {$f$}
node[near end,above] {$f$}
node[pos=.25,above] {$f$}
node[pos=.75,above] {$f$}
```

A curved labeled connection is

Code

```
\draw[-{Stealth}]
  (A)
  to[bend left=25]
```

Code

```
node[above] {$f$}  
(B);
```

For modern arrow tips, load

Code

```
\usetikzlibrary{arrows.meta}
```

D.6 Line, Shape, Fill, and Transparency Options

Frequently used path and node options include

Purpose	Typical options
Thickness	thick, very thick, line width=1.2pt
Pattern of line	dashed, dotted
Rounded corners	rounded corners, rounded corners=4pt
Border and fill	draw, draw=blue, fill=yellow!20
Node shapes	circle, rectangle, or a shape supplied by a loaded library
Minimum size	minimum width=20mm, minimum height=8mm, minimum size=8mm
Opacity	opacity=.4, draw opacity=.5, fill opacity=.2, text opacity=1

A filled and outlined path can be written

Code

```
\filldraw[fill=blue!15,draw=blue]  
(0,0) circle (8mm);
```


D.7 Reusable Styles

Define styles with `\tikzset`.

Code

```
\tikzset{
  vertex/.style={
    draw,
    circle,
    minimum size=8mm
  },
  important vertex/.style={
    vertex,
    very thick,
    fill=yellow!20
  }
}
```

Use a style by name.

Code

```
\node[vertex] (A) at (0,0) {$A$};
\node[important vertex] (B) at (3,0) {$B$};
```

Append or prefix options to an existing style with

Code

```
\tikzset{
  vertex/.append style={fill=blue!10}
}
```

D.8 Styles with Parameters

One parameter can be introduced with

Code

```
\tikzset{
  colored vertex/.style={
    draw,
    circle,
    fill=#1
  }
}

\node[colored vertex=blue!20] {$A$};
```

A default value uses

Code

```
\tikzset{
  colored vertex/.style={fill=#1},
  colored vertex/.default=blue!20
}
```

Several positional parameters can be parsed with

Code

```
\tikzset{
  labeled vertex/.style args={#1/#2}{
    draw,
    circle,
    fill=#1,
    label=above:{#2}
  }
}
```

D.9 Custom Keys

A simple executable key is

Code

```
\tikzset{
  announce/.code={
    \node at (0,0) {#1};
  }
}
```

A value can be stored in a macro.

Code

```
\tikzset{
  /my diagram/radius/.store in=\myradius,
  /my diagram/radius/.initial=1cm
}
```

Move to a key directory with

Code

```
\tikzset{
  /my diagram/.cd,
  radius=2cm
}
```

Choice keys use

Code

```
\tikzset{
  /my diagram/mode/.is choice,
  /my diagram/mode/plain/.code={...},
  /my diagram/mode/emphasis/.code={...}
}
```

D.10 Loops and Computed Values

A simple loop is

Code

```
\foreach \x in {0,1,2,3,4,5}
  \draw (\x,0) -- (\x,0.3);
```

Several variables can be supplied by paired data.

Code

```
\foreach \x/\name in {0/A,2/B,4/C}
  \node at (\x,0) {\name};
```

Nested loops have the form

Code

```
\foreach \i in {1,...,4}{
  \foreach \j in {1,...,3}{
    \fill (\i,\j) circle (1pt);
  }
}
```

Compute a reusable value with

Code

```
\pgfmathsetmacro{\angle}{360/7}
```

D.11 Scopes and Transformations

Limit transformations and local options with a scope.

Code

```
\begin{scope}[xshift=3cm,rotate=20]  
  \draw (0,0) rectangle (2,1);  
\end{scope}
```

Frequently used transformations are

Code

```
xshift=1cm  
yshift=-5mm  
shift={(1,2)}  
rotate=30  
scale=1.2  
xscale=1.5  
yscale=.8
```

When node shapes should transform with the surrounding picture, use

Code

```
transform shape
```

D.12 Clipping and Layers

Clip subsequent drawing to a region with

Code

```
\begin{scope}  
  \clip (0,0) rectangle (4,3);  
  % drawing restricted to the rectangle  
\end{scope}
```

For the convenient background layer, load

Code

```
\usetikzlibrary{backgrounds}
```

and use

Code

```
\begin{scope}[on background layer]  
  \fill[gray!15] (-.5,-.5) rectangle (4.5,2.5);  
\end{scope}
```

For explicitly declared layers, use the PGF layer commands introduced in Chapter 24.

D.13 Reusable Pictures with pic

A minimal pic definition is

Code

```
\tikzset{
  pics/target/.style={
    code={
      \draw (0,0) circle (5mm);
      \draw (0,0) circle (2.5mm);
      \fill (0,0) circle (1.5pt);
    }
  }
}
```

Place it with

Code

```
\pic {target};
\pic at (3,0) {target};
```

Name pic instances when their internal named objects must be referenced.

Code

```
\pic (A) at (0,0) {module};
\pic (B) at (4,0) {module};
```

A pic is appropriate when the repeated object has its own internal geometry. A style is appropriate when several existing objects merely share appearance or behavior.

D.14 Matrices and Trees

Load the matrix library when using matrix-specific features.

Code

```
\usetikzlibrary{matrix}
```

A simple matrix of nodes is

Code

```
\matrix[matrix of nodes,  
  row sep=4mm,  
  column sep=6mm] {  
  $a$ & $b$ \\  
  $c$ & $d$ \\  
};
```

A simple tree is

Code

```
\node {Root}  
  child {node {Left}}  
  child {node {Right}};
```

Set tree spacing with options such as

Code

```
level distance=12mm,  
sibling distance=20mm
```


D.15 Intersections

Load

Code

```
\usetikzlibrary{intersections}
```

Name the paths to be intersected.

Code

```
\path[name path=lineA] (0,0) -- (4,3);  
\path[name path=lineB] (0,3) -- (4,0);
```

Compute and name the intersection.

Code

```
\path[  
  name intersections={  
    of=lineA and lineB,  
    by=P  
  }  
];
```

Then use (P) like any other named coordinate.

D.16 Decorations

Path-morphing decorations such as coils and zigzags require

Code

```
\usetikzlibrary{decorations.pathmorphing}
```

A typical pattern is

Code

```
\draw[
  decorate,
  decoration={
    coil,
    amplitude=3pt,
    segment length=6pt
  }
] (0,0) -- (4,0);
```

Braces require

Code

```
\usetikzlibrary{decorations.pathreplacing}
```

and may be drawn with

Code

```
\draw[
  decorate,
  decoration={brace,amplitude=5pt}
] (0,0) -- (4,0);
```

D.17 Externalization

Load and activate externalization with

Code

```
\usetikzlibrary{external}  
\tikzexternalize
```

Place generated files in a dedicated directory with

Code

```
\tikzexternalize[prefix=tikz-cache/]
```

Give the next picture a stable name with

Code

```
\tikzsetnextfilename{triangle-median}
```

Useful controls include

Code

```
\tikzexternaldisable  
\tikzexternalenable  
\tikzset{external/remake next}  
\tikzset{external/force remake}
```

Externalization is a production technique. Debug the picture normally before adding the externalized build process.

D.18 Libraries Used Most Often in This Book

Library	Use it for
<code>arrows.meta</code>	Modern arrow tips such as <code>Stealth</code> .
<code>positioning</code>	Relative node placement such as <code>right=of</code> .
<code>calc</code>	Midpoints, ratios, shifts, and calculated coordinates.
<code>matrix</code>	Matrices, grids, and structured layouts of nodes.
<code>intersections</code>	Intersections of named paths.
<code>decorations.</code>	Coils, snakes, waves, and zigzags.
<code>pathmorphing</code>	
<code>decorations.</code>	Braces and path-replacement decorations.
<code>pathreplacing</code>	
<code>patterns</code>	Pattern fills.
<code>backgrounds</code>	Convenient background-layer drawing.
<code>fit</code>	Nodes enclosing other nodes or coordinates.
<code>external</code>	Caching TikZ pictures as external graphics.

For the larger inventory and the distinction between TikZ libraries and external packages, see Appendix B.

D.19 Task-to-Syntax Quick Finder

I want to ...	Start with ...	Chapter
Draw a path	<code>\draw (A) -- (B);</code>	1
Place a node	<code>\node ... {text};</code>	2
Connect named nodes	<code>\draw (A) -- (B);</code>	3
Reuse an appearance	<code>name/.style={...}</code>	2, 18–20
Place nodes relatively	<code>right=of A</code>	6
Build a tree	<code>child {node ...}</code>	7
Compute a midpoint	<code>\$(A)!0.5!(B)\$</code>	8
Draw a curved connection	<code>to[bend left=...]</code>	9
Use modern arrow tips	<code>-{Stealth}</code>	10
Transform a drawing	<code>scope, shift, rotate, scale</code>	11
Lay out nodes in a matrix	<code>\matrix</code>	12
Draw a coil or brace	<code>decorate, decoration=...</code>	13
Clip or use transparency	<code>\clip, opacity=</code>	14
Define a configurable key	<code>/.code, /.store in</code>	21
Reuse a graphical component	<code>pics/name/.style, \pic</code>	22
Generate repeated geometry	<code>\foreach, \pgfmathsetmacro</code>	23
Draw behind the main objects	<code>on background layer</code>	24
Externalize finished figures	<code>\tikzexternalize</code>	25
Compute path intersections	<code>name path, name intersections</code>	26

Continued on next page

I want to ...	Start with ...	Chapter
Control node attachment	anchors such as east , base , north west	27
Organize a large diagram	styles, pics, loops, calculated coordinates, layers	28
Debug and maintain a project	reduce, test, restore one feature at a time	29

D.20 Final Advice

The most useful TikZ syntax is not the longest or most sophisticated form. It is the form that makes the mathematical relationship visible. Name important objects, use styles for repeated appearance, use pics for repeated geometry, compute coordinates from relationships, and load libraries deliberately.

Principle

Use this appendix to recover syntax quickly. Then return to the source and make the code say what the diagram means.